evaluator.py — technical

src/moe_pruning/evaluation/evaluator.py · 215 lines · moe_pruning.evaluation.evaluator

Purpose

Example preparation and caching, the evaluation loop, and the exact per-layer reconstruction probe.

Formulas

```
y_orig(t) = sum over original top-k slots of g_orig(t,e) * E_e(x_t) (original routing, original experts)

mse_per_token = sum_t ||y_orig(t) - y_pruned(t)||^2 / N_real

rel_error_all = sum_t ||y_orig - y_pruned||^2 / sum_t ||y_orig||^2; rel_error_affected = same over tokens with a removed expert in their original top-k
```

Step by step

Classes and functions in source order. Signatures, line ranges, raised exceptions and settings are extracted from the code; numbered steps describe what the code does.

`eval_settings`

```
def eval_settings(cfg: ExperimentConfig, task: TaskObjective) -> dict
```

lines 27-34 · settings EVALUATION_DATASET_PATH, EVAL_MAX_SAMPLES, MODEL_PATH or HF_MODEL_ID

Settings that determine the evaluation examples (dataset checksum + task settings).

1. Dataset path, SHA-256 of the evaluation file, `EVAL_MAX_SAMPLES`, tokenizer source, plus `task.eval_settings` (span-corruption parameters for seq2seq, max length for causal).

`prepare_eval_examples`

```
def prepare_eval_examples(cfg: ExperimentConfig, tokenizer, paths: ArtifactPaths, task: TaskObjective) -> tuple[list, dict]
```

lines 37-56 · settings EVALUATION_DATASET_PATH, EVAL_MAX_SAMPLES

Build the evaluation examples once and cache them; every stage reuses the cache.

1. Compute settings and their fingerprint.
2. If `eval_examples.pt` exists with the same fingerprint, return the cached examples.
3. Otherwise load documents, build one example per document with `task.build_eval_example`, drop examples shorter than 2 tokens, record counts, and `torch.save` examples + settings.

`class ReferenceExpertProbe`

lines 73-141 · dataclass

Measures ||y_original - y_pruned||^2 of pruned layers on the evaluation data.

Field	Type	Default
`model`	nn.Module	
`adapter`	MoEAdapter	
`layers`	dict[str, MoELayerInfo]	
`reference_experts`	dict[str, dict[int, nn.Module]]	
`acc`	dict[str, _ProbeAccumulator]	field(default_factory=dict)

`ReferenceExpertProbe.original_layer_output`

```
def original_layer_output(self, layer: MoELayerInfo, moe_input: torch.Tensor) -> tuple[torch.Tensor, torch.Tensor]
```

lines 96-109

(y_original [B,S,D] float32, original expert indices [B,S,K]).

1. `adapter.route(..., apply_mask=False)` gives the original indices and gates `[B, S, K]` (any reroute mask ignored).

2. For each distinct expert id present: tokens routed to it in any slot; its gate per token = sum over slots of gate * (index == e) (an expert appears at most once per token).
3. Expert module = original reference module if the expert was removed, otherwise the model's own (retained) expert.
4. Accumulate `g * E(x)` in float32 into `y_orig`.

`ReferenceExpertProbe.__call__`

```
def __call__(self, record: MoEForwardRecord, token_mask: torch.Tensor) -> None
```

lines 111-125

Add one layer forward record to the error sums.

1. Called from the adapter instrumentation for each pruned layer forward with that stack's real-token mask.
2. Compute `y_orig` and original indices; affected tokens = real tokens with a removed expert among their original top-k.
3. Add squared error and original squared norm (all real tokens and affected tokens) to the layer's accumulator.

`ReferenceExpertProbe.summary`

```
def summary(self) -> dict
```

lines 127-141

Per-layer MSE and relative errors (all tokens / affected tokens).

`build_reference_probe`

```
def build_reference_probe(model: nn.Module, adapter: MoEAdapter, layers: dict[str, MoELayerInfo],
replaced: dict[str, list[int]], reader, device: torch.device) -> ReferenceExpertProbe
```

lines 144-160

Probe with the original weights of the removed experts (read lazily via `reader`).

1. For each pruned layer and removed expert: `adapter.load_original_expert(reader, ...)` reads only that expert's tensors from the original checkpoint.
2. Move it to the device of that layer's router (correct GPU when sharded) and the model dtype; eval mode.

`class EvalOutput`

lines 167-173 · dataclass

Metrics plus per-example losses, token counts and predictions of one evaluation.

Field	Type	Default
`metrics`	dict	
`example_ids`	list[int]	
`loss_sums`	list[float]	
`token_counts`	list[int]	
`predictions`	list[list[int]]	
`layer_probe`	dict | None	None

`evaluate_model`

```
def evaluate_model(model: nn.Module, examples: list, device: torch.device, batch_size: int,
pad_token_id: int, task: TaskObjective, adapter: MoEAdapter | None=None, probe: ReferenceExpertProbe |
None=None, desc: str='evaluating') -> EvalOutput
```

lines 177-215

Run the model over all examples and collect losses, predictions and probe errors.

1. Optionally enter `adapter.instrument` for the probed layers; the callback reads the current batch's masks.
2. For each batch from `task.iter_batches`: move inputs to the input device, forward, `assert_finite(logits)`.

3. `task.token_nll` -> per-example NLL sum and token count; `task.predictions` -> argmax at scored positions.
4. After the loop, `summarize_losses` (perplexity only if the task reports it) and the probe summary.